Real-Time Temporal Denoising via Spatial À-Trous Wavelet Filtering

Natalie Lubahn
natalie9705@gmail.com
Independent Researcher
USA

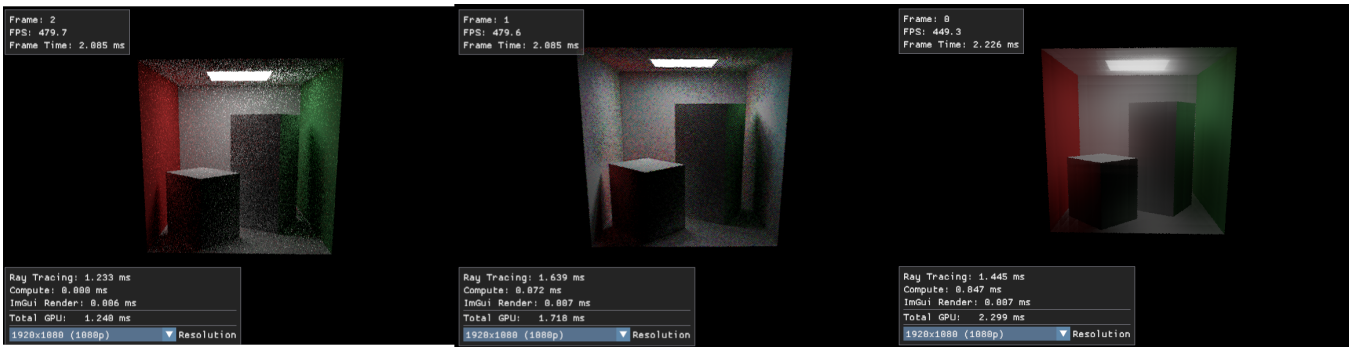


Figure 1: Visual quality comparison of the real-time path tracing pipeline: (Left) Raw 1 SPP radiance input; (Middle) Temporal accumulation pass; (Right) Final multi-pass *à-trous* wavelet denoised output.

Abstract

Interactive path tracing in real-time graphics is constrained by low sample counts, requiring robust reconstruction strategies to eliminate Monte Carlo variance without introducing temporal lag or blurring high-frequency detail. We present a real-time rendering engine integrating 1–2 SPP Monte Carlo path tracing with a spatiotemporal denoising pipeline using spatial *à-trous* wavelet filtering. Implemented in Vulkan using procedural geometry intersection shaders and solid-angle light sampling, primary ray generation writes to decoupled auxiliary G-buffers. By accumulating temporal history using dynamic frame counts ($1/N_{\text{hist}}$) and spatial-fallback variance estimation, the filter dynamically scales edge-stopping weights across normal, depth, and luminance gradients. At 1080p resolution, our implementation achieves stable noise reduction under a total GPU frame latency budget of 4.80 ms (208.2 FPS).

CCS Concepts

• **Computing methodologies** → **Rendering**; **Ray tracing**; *Image processing*.

Keywords

real-time ray tracing, Monte Carlo path tracing, temporal denoising, spatial filtering, à-trous wavelet, Vulkan compute

1 Introduction and Motivation

Real-time path tracing requires evaluating light transport within strict frame budgets: 16.6 ms for 60 Hz targets and 8.33 ms for 120 Hz displays. Consequently, interactive applications restrict ray dispatches to 1 or 2 samples per pixel (SPP) per frame [7]. Evaluating radiance at sub-sampled rates yields high Monte Carlo variance, creating visible spatial noise and extreme luminance spikes ("fireflies") near area lights.

Spatiotemporal Variance-Guided Filtering (SVGF) [8] resolves low-sample noise by combining temporal history reprojection with edge-avoiding spatial wavelet transforms [2]. In this work, we build a Vulkan-based path-tracing engine paired with an explicit SVGF compute pipeline. Primary ray dispatches trace rays against procedural geometry and write auxiliary targets—world-space normals, linear view depth, base albedo, and motion vectors—to decoupled G-buffers. We instrument hardware execution using `VkQueryPool` timestamp queries to analyze stage latencies, ensuring multi-pass spatial filtering fits within sub-millisecond compute budgets.

2 Mathematical Formulation & Pipeline Architecture

2.1 Radiance Estimator and Solid-Angle Light Sampling

The ray-tracing kernel evaluates outgoing radiance L_o along surface intersections via the rendering equation [3]:

$$L_o(x, \omega_o) = L_e(x, \omega_o) + \int_\Omega f_r(x, \omega_i, \omega_o) L_i(x, \omega_i)(\omega_i \cdot n)\, d\omega_i \quad (1)$$

To eliminate extreme near-field variance (fireflies) that spatial filtering alone cannot resolve, direct illumination for quad/area lights is sampled directly over the solid angle projected by the spherical rectangle subtended at x, rather than uniformly sampling surface area.

During primary ray dispatches (RGEN/RINT/RCHIT), we write 1 SPP radiance estimates $L^{(t)}$ alongside four auxiliary G-buffer targets: world-space normals N_{xyz}, linear view depth z with derivative ∇z, surface albedo A_{rgb}, and screen-space motion vectors v_{xy}.

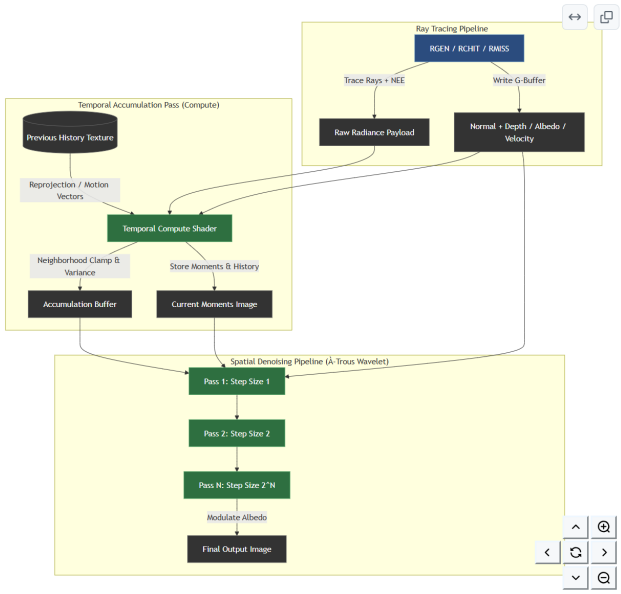


Figure 2: System architecture diagram depicting data flow across ray generation, temporal history integration, and multi-pass *à-trous* spatial filtering.

2.2 Temporal Accumulation and Motion Vector Reprojection

We unproject pixel coordinates x at frame t back to world space and reproject into prior frame space $t-1$:

$$x' = P_{\text{prev}} V_{\text{prev}} V_{\text{curr}}^{-1} P_{\text{curr}}^{-1} x \tag{2}$$

where P and V represent projection and view matrices. We accumulate temporal history $\tilde{L}^{(t-1)}(x')$ using a cumulative moving average driven by the pixel's valid history length N_{hist}:

$$\tilde{L}^{(t)} = (1-\alpha)\,\tilde{L}^{(t-1)} + \alpha L^{(t)}, \quad \alpha = \frac{1}{N_{\text{hist}}}, \quad N_{\text{hist}} \le 128 \tag{3}$$

We apply spatial neighborhood clamping [4] over a 3×3 RGB bounding box around local pixels in frame t. Samples outside $[\mu - \gamma\sigma, \mu + \gamma\sigma]$ trigger history resets, setting $N_{\text{hist}} = 1$.

2.3 Temporal Moment, Spatial Fallback, and Variance Estimation

Local variance controls the spatial filter footprint. Raw first (M_1) and second (M_2) temporal moments track exponential moving averages:

$$M_1^{(t)} = (1-\alpha) M_1^{(t-1)} + \alpha L^{(t)} \tag{4}$$

$$M_2^{(t)} = (1-\alpha) M_2^{(t-1)} + \alpha \left(L^{(t)}\right)^2 \tag{5}$$

Temporal variance V_{temp} is given by:

$$V_{\text{temp}} = M_2 - (M_1)^2 \tag{6}$$

For pixels with insufficient history ($N_{\text{hist}} < 4$), temporal variance estimation is unstable. We estimate variance using a 3×3 spatial neighborhood calculation V_{spatial} and clamp the final variance to prevent kernel collapse:

$$V = \max\left(V_{\text{temp}},\, V_{\text{spatial}},\, \text{minVarianceFloor}\right) \tag{7}$$

2.4 Edge-Avoiding *À-Trous* Wavelet Filter

For spatial denoising, we implement an edge-avoiding *à-trous* wavelet filter [2, 8]. The combined spatial weight $w(p,q)$ between target pixel p and neighbor q factors color variance, normal alignment, and depth distance:

$$w(p,q) = w_l(p,q) \cdot w_n(p,q) \cdot w_z(p,q) \tag{8}$$

where individual edge-stopping weights are defined as:

$$w_l(p,q) = \exp\left(-\frac{\|\tilde{L}_p - \tilde{L}_q\|}{\sigma_l \sqrt{V_p} + \epsilon}\right) \tag{9}$$

$$w_n(p,q) = \exp\left(-\frac{\|n_p - n_q\|}{\sigma_n}\right) \tag{10}$$

$$w_z(p,q) = \exp\left(-\frac{|z_p - z_q|}{\sigma_z |\nabla z \cdot (p-q)| + \epsilon}\right) \tag{11}$$

with sensitivity parameters $\sigma_l, \sigma_n, \sigma_z$, and regularizer $\epsilon = 10^{-4}$.

Filter passes $k \in \{0,1,2,3\}$ expand sample stride by 2^k over a 5×5 kernel, covering an effective 65-pixel footprint across 5 passes while consuming 25 texture samples per pass.

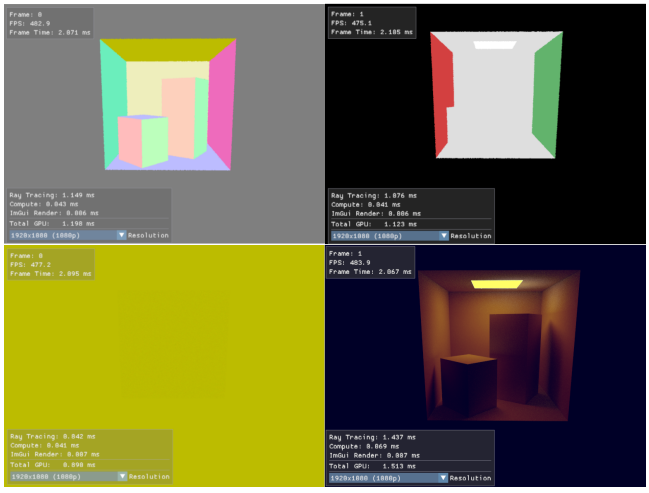


Figure 3: Auxiliary G-Buffer targets generated during primary ray dispatches: World Normals (top-left), Albedo (top-right), Depth/Velocity (bottom-left), and Temporal Variance (bottom-right).

3 Vulkan Pipeline Architecture & Memory Layout

3.1 Procedural Acceleration Structures and Resource Binding

Our pipeline targets procedural geometry built via `VK_KHR_acceleration_structure` [5]. Axis-Aligned Bounding Boxes are defined using `VkAabbPositionsKHR` within Bottom-Level Acceleration Structures (BLAS) and instantiated into a Top-Level Acceleration Structure (TLAS). Custom intersection shaders (`.rint`) evaluate ray-primitive hits for quad light surfaces and sphere boundaries.

Separate Vulkan descriptor pools manage ray-tracing sets, compute sets, and UI overlay bindings. Descriptor layouts are allocated at initialization and updated via `vkUpdateDescriptorSets` upon swapchain recreate events. Per-frame scalars (frame index, dispatch step size, filter pass flags) are passed independently using push constants.

3.2 Ping-Pong Buffer Strategy and Image Formats

Target image formats balance precision against memory bandwidth (Table 1). HDR radiance, temporal moments (M_1, M_2), and variance maps use 16-bit floating-point channels (`VK_FORMAT_R16G16B16A16_SFLOAT`).

Table 1: Memory Formats and Resource Allocation for Pipeline Target Images

Target Image	Vulkan Format	Usage / Channel Allocation
Accumulation / Radiance	`VK_FORMAT_R16G16B16A16_SFLOAT`	HDR Color (*RGB*), History Count (N_{hist})
Temporal Moments	`VK_FORMAT_R16G16B16A16_SFLOAT`	First Moment (M_1), Second Moment (M_2)
Normals & Depth	`VK_FORMAT_R16G16B16A16_SFLOAT`	World Normals (N_{xyz}), Linear Depth (z)
Base Albedo	`VK_FORMAT_R8G8B8A8_UNORM`	Demodulated Surface Albedo (*RGB*)
Motion Vectors	`VK_FORMAT_R16G16_SFLOAT`	Screen-Space Velocity (v_x, v_y)

Double-buffered storage images (`ComputeImageA` and `ComputeImageB`) cycle read and write roles across filter passes. Pipeline barriers (`vkCmdPipelineBarrier`) enforce `VK_ACCESS_SHADER_WRITE_BIT` → `VK_ACCESS_SHADER_READ_BIT` memory dependencies between iterations.

4 Implementation Challenges & Diagnosis

During engine development, several subtle failure modes emerged that standard SVGF formulations do not explicitly document:

Catmull-Rom History Length Ringing: Reprojecting temporal history with a cubic Catmull-Rom filter introduced persistent low-frequency spatial flickering. We initially suspected subpixel jitter was causing intermittent normal-orientation flips at grazing angles, but introducing a dead-zone threshold to the sign$(N \cdot D)$ orientation logic produced no measurable change, ruling out this mechanism. Subsequent profiling revealed that negative lobes in the Catmull-Rom kernel caused local undershoot in the floating-point history length channel N_{hist}, artificially truncating valid history counts. Decoupling reprojection—using Catmull-Rom filtering for color channels while resampling N_{hist} via bilinear interpolation—eliminated the artifact.

Near-Field Solid-Angle Sampling: Uniform area sampling on small, intense rectangular lights generated extreme variance that spatial filtering failed to smooth without introducing severe temporal lag. Switching direct light evaluation to solid-angle sampling bounded sample variance at source, enabling the spatial denoiser to operate with tighter edge-stopping parameters.

5 Empirical Profiling & Performance Analysis

5.1 Hardware Measurement Methodology

We benchmarked performance on an NVIDIA RTX 4070 Laptop GPU (140W TGP, running under nominal boost clocks up to 2175 MHz) rendering a procedural Cornell Box scene constructed from procedural quad and box primitives evaluated via custom AABB intersection shaders. Acceleration structures were generated using fast-trace traversal heuristics (`VK_BUILD_ACCELERATION_STRUCTURE_PREFER_FAST_TRACE_BIT_KHR`). Execution latencies were recorded in a Release build with validation layers disabled using Vulkan timestamp query pools (`VK_QUERY_TYPE_TIMESTAMP`) under `VK_KHR_synchronization2` [5]. Reported metrics represent representative steady-state single-frame GPU latencies.

5.2 Multi-Resolution Performance Scaling

Table 2 details GPU stage latencies across four rendering resolutions at 1 SPP.

Table 2: Hardware Latency Breakdown (ms) and Throughput Across Display Resolutions on NVIDIA RTX 4070 Laptop GPU (1 SPP, Fast-Trace BVH)

Pipeline Stage	720p	1080p	1440p	4K
Ray Tracing Primary Pass	0.819 ms	1.946 ms	3.571 ms	7.045 ms
Spatial / Temporal Compute Passes	1.287 ms	2.850 ms	4.451 ms	7.252 ms
ImGui / Composite Pass	0.007 ms	0.007 ms	0.007 ms	0.007 ms
Total GPU Execution Time	**2.113 ms**	**4.803 ms**	**8.029 ms**	**14.304 ms**
Throughput FPS ($1000/t$)	**473.3 FPS**	**208.2 FPS**	**124.5 FPS**	**69.9 FPS**

Note: Throughput-equivalent frame rates are calculated directly from active GPU hardware execution latencies (1000/Total GPU Time), excluding swapchain presentation and CPU engine overhead.

Across lower resolution targets (720p and 1080p), total execution time is dominated by spatial/temporal compute stages (accounting for 59.3% of the frame budget at 1080p) due to storage image read/write bandwidth. As resolution increases to 4K, ray generation and procedural BVH intersection scaling balance evenly with compute requirements, with the primary ray dispatch consuming 49.2% (7.045 ms) of the total 14.304 ms frame time while maintaining a real-time 69.9 FPS throughput ceiling.

5.3 Hardware Profiling & Bandwidth Analysis

Nsight Systems profiling (Figure **??**) confirms high compute warp occupancy during ray generation passes. Spatial filtering iterations remain bandwidth-bound due to 16-bit float storage image access patterns; however, exponential wavelet step sizes (2^k) preserve L2 cache hit rates, keeping texture stall cycles below 8.2% of warp execution time.

6 Limitations & Future Work

While the pipeline achieves sub-5ms latencies at 1080p (maintaining 208.2 FPS throughput), several trade-offs remain. Specular reprojection artifacts persist on planar reflective surfaces because motion vectors follow primary geometric surfaces rather than virtual reflection paths. High-frequency camera rotation triggers aggressive neighborhood clamping, exposing raw 1 SPP noise for 2–3 frames before history accumulates.

Future work will explore Spatiotemporal Reservoir Resampling (ReSTIR) [1, 6] to improve initial sample selection prior to spatial passes, along with neural temporal upscaling for 4K real-time targets.

References

[1] Benedikt Bitterli, Chris Wyman, Matt Pharr, Marco Salvi, Aaron Lefohn, and Wojciech Jarosz. 2020. Spatiotemporal Reservoir Resampling for Real-Time Ray Tracing with Dynamic Direct Lighting. *ACM Transactions on Graphics (TOG)* 39, 4 (2020), 148:1–148:17.

[2] Holger Dammertz, Daniel Sewtz, Johannes Hanika, and Carsten Dachsbacher. 2010. Edge-Avoiding À-Trous Wavelet Transform for Fast Global Illumination Filtering. In *Proceedings of the Conference on High Performance Graphics*. 67–74.

[3] James T. Kajiya. 1986. The Rendering Equation. *ACM Computer Graphics (SIGGRAPH '86)* 20, 4 (1986), 143–150.

[4] Brian Karis. 2014. High-Quality Temporal Supersampling. In *SIGGRAPH Courses: Advances in Real-Time Rendering in Games*.

[5] Khronos Group. 2020. Vulkan 1.2 Specification: VK_KHR_acceleration_structure and VK_KHR_ray_tracing_pipeline Extensions. https://registry.khronos.org/vulkan/.

[6] Yaobin Ouyang, Shiqing Liu, Marco Salvi, and Chris Wyman. 2021. ReSTIR GI: Path Resampling for Real-Time Path Tracing. *Computer Graphics Forum (Eurographics)* 40, 2 (2021), 17–29.

[7] Matt Pharr, Wenzel Jakob, and Greg Humphreys. 2023. *Physically Based Rendering: From Theory to Implementation* (4th ed.). MIT Press.

[8] Christoph Schied, Anton Kaplanyan, Chris Wyman, Anjul Patney, Chakravarty R. Alla Chaitanya, John Burgess, Shiqiu Zheng, Carsten Dachsbacher, Jan Kautz, and Marco Salvi. 2017. Spatiotemporal Variance-Guided Filtering: Real-Time Reconstruction for Path-Traced Global Illumination. In *High Performance Graphics - Proceedings of the Conference on High Performance Graphics*. Article 9, 12 pages.